salesianos
COLEGIO SAN IGNACIO
CÁDIZ

Entornos de Desarrollo

UD5. Diseño y Realización de Pruebas (1. Introducción)

Índice

- Introducción
- ¿Qué es probar?
- Imposibilidad de prueba exhaustiva
- Tipos de pruebas
- Otros conceptos
- Otras pruebas

Introducción

- **Fase de pruebas** → Una de las últimas fases del ciclo de vida antes de entregar el producto
- Sorpresa nuevos programadores → **enorme cantidad de tiempo y esfuerzo** que requiere esta fase. Se estima que la mitad del esfuerzo de desarrollo de un programa (tanto en tiempo como en gastos) se va en esta fase.
- Programas que involucran vidas humanas (medicina, equipos nucleares, etc.) el costo de la fase de pruebas puede fácilmente superar el 80%.
- Pese a su enorme impacto en el coste de desarrollo, es una fase que muchos programadores aún consideran clasificable como un arte y, por tanto, como **difícilmente conceptualizable**.

¿Qué es probar?

- ¿Por qué probar? Fallos software → graves pérdidas económicas; éstos son 100 a 1000 veces más costosos de encontrar y reparar después de la construcción.

- Los **objetivos** fundamentales de esta etapa son:
  - Evitar plazos y presupuestos incumplidos, insatisfacción del usuario, escasa productividad y mala calidad en el software producido y finalmente la pérdida de clientes.
  - Automatizar el proceso de pruebas (consigue reducciones de hasta un 75% en el costo de la fase de mantenimiento)

- **Probar un programa** es ejercitarlo con la peor intención a fin de encontrarle fallos, *descubrir el mayor número de errores* posibles en un programa.

¿Qué es probar?

- Todos los programas tienen errores y la fase de pruebas los descubre. **El objetivo específico de la fase de pruebas es encontrar cuantos más errores, mejor.**

- Es frecuente encontrarse con el error de afirmar que el objetivo de esta fase es convencerse de que el programa funciona bien. En realidad ese es el objetivo propio de las fases anteriores (¿quién va a pasar a la sección de pruebas un producto que sospecha que está mal?).

- Probar un programa es buscarle fallos a “mala leche”, siendo la finalidad poner en evidencia al programa.

Imposibilidad de prueba exhaustiva

- **¿Por qué no probar todo?**
- Prueba ideal → **Exponerlo en todas las situaciones posibles**, así encontraríamos hasta el último fallo → Esto es **imposible** desde todos los puntos de vista: humano, económico e incluso matemático.
- Todo es finito en programación → el número de pruebas posibles es finito (se excluyen bucles)
- Número de combinaciones posibles muy grande → Imposible su identificación y ejecución a todos los efectos prácticos.
- **Ejemplo**: Compilador de cualquier lenguaje: ¡habría que escribir, compilar y ejecutar todos y cada uno de los programas que se pudieran escribir con dicho lenguaje!
- No podemos alcanzar la perfección, pero algo habrá que hacer.
- Buscar formas humanamente abordables y económicamente aceptables de encontrar errores.

Tipos de pruebas (Categorización)

- **Qué conocemos**:
  - Pruebas de **caja negra (Pruebas Funcionales)**: no conocemos la implementación del código, sólo la interfaz.
  - Pruebas de **caja blanca (Pruebas Estructurales)**: conocemos el código (la implementación de éste) que se va a ejecutar y podemos definir las pruebas que cubran todos los posibles caminos del código.

Tipos de pruebas (Categorización)

- Grado de **automatización**:
  - Pruebas **manuales**: son las que se hacen normalmente al programar o las que ejecuta una persona con la documentación generada durante la codificación (P. ej.- comprobar cómo se visualiza el contenido de una página web en dos navegadores diferentes).
  - Pruebas **automáticas**: se usa un determinado software para sistematizar las pruebas y obtener los resultados de las mismas (P. ej.- verificar un método de ordenación).

Tipos de pruebas (Categorización)

- En función de **qué se prueba**:
  - Pruebas **unitarias**: Componente del software (función, clase, librería, etc.)
  - Pruebas de **integración**: Construir el sistema a partir de los distintos componentes y probarlo con todos integrados. Estas pruebas deben realizarse progresivamente.
  - Pruebas **funcionales**: sobre el sistema funcionando se comprueba que cumple con la especificación (normalmente a través de los casos de uso).

Otros conceptos

- **Completitud**: Grado de fiabilidad de las pruebas y por consiguiente la fiabilidad del software.
- No es posible llegar al 100% puesto que nunca llegaremos a realizar todas las pruebas posibles al software, puesto que las pruebas tienen un coste (P. ej.- Bug del Excel).
- **Depuración**: ejecución controlada del software que nos permite corregir un error (P. ej.- Usar el depurador de Visual Studio).

Otras pruebas

- Pruebas de **aceptación**: Pruebas funcionales que realiza el cliente antes de poner la aplicación en producción.
  - Pruebas **alfa**: Se invita al cliente al entorno de desarrollo, trabajando sobre un entorno controlado.
  - Pruebas **beta**: Se desarrollan en el entorno del cliente, que se queda solo con el producto en un entorno sin controlar.
- Ambas pruebas son habituales cuando se va a distribuir el programa a **muchos clientes**
- El cliente final decide qué pruebas va a aplicarle al producto antes de darlo por bueno y pagarlo.

Otras pruebas

- **Regresión (regression testing):**
- Evolución de sistemas
- Comprobar que en las nuevas versiones de los sistemas (modificaciones realizadas) no provocan errores donde antes no los había.
- **Recorridos (walkthroughs)** :
- Más aplicada en control de calidad.
- Sentar alrededor de una mesa a los desarrolladores y a una serie de críticos, bajo las órdenes de un moderador
- Los revisores se leen el programa línea a línea y piden explicaciones de todo lo que no está meridianamente claro.

Otras pruebas

- **Aleatorias (random testing)**
- La probabilidad de descubrir un error es prácticamente la misma si se hacen una serie de pruebas aleatoriamente elegidas, que si se hacen siguiendo las instrucciones dictadas por criterios de cobertura (caja negra o blanca).
- Razonable comenzar la fase de pruebas con una serie de casos elegidos al azar. Esto pondrá de manifiesto los errores más patentes.
- Si el programa es poco crítico (una aplicación personal, un juego, ...) puede que esto sea suficiente. Pero si se trata de una aplicación militar o con riesgo para vidas humanas, es claramente insuficiente.

Otras pruebas

- **Solidez (robustness testing)**
- Capacidad del sistema para salir de situaciones embarazosas provocadas por errores en el suministro de datos. Sistemas con una interfaz al exterior, en particular cuando la interfaz es humana.
- Por ejemplo, en un sistema que admite una serie de órdenes (commands) se deben probar algunos de los siguientes extremos:
  - órdenes correctas, todas y cada una
  - órdenes con defectos de sintaxis, tanto pequeñas desviaciones como errores de bulto
  - órdenes correctas, pero en orden incorrecto, o fuera de lugar
  - la orden nula (línea vacía, una o más)
  - órdenes correctas, pero con datos de más

Otras pruebas

- **Aguante (stress testing)**
- En ciertos sistemas es conveniente saber hasta dónde aguantan, bien por razones internas (¿hasta cuantos datos podrá procesar?), bien externas (¿es capaz de trabajar con un disco al 90%?, ¿aguanta una carga de la CPU del 90?, etc.)
- **Prestaciones (performance testing)**
- A veces es importante el tiempo de respuesta, u otros parámetros de gasto. Típicamente nos puede preocupar cuánto tiempo le lleva al sistema procesar tantos datos, o cuánta memoria consume, o cuánto espacio en disco utiliza, o cuántos datos transfiere por un canal de comunicaciones, etc... Para todos estos parámetros suele ser importante conocer cómo evolucionan al variar la dimensión del problema (por ejemplo, al duplicarse el volumen de datos de entrada).

Otras pruebas

- **Mutación (mutation testing)**
- Es una técnica curiosa consistente en alterar ligeramente el sistema bajo pruebas (introduciendo errores) para averiguar si nuestra batería de pruebas es capaz de detectarlo. Si no, más vale introducir nuevas pruebas. Todo esto es muy laborioso.
- **Conformidad u Homologación** ***(conformance testing)***
- Normas a las que el programa debe atenerse
- Son pruebas de caja negra para comprobar que el programa se ajusta a las normas debidas
- **Interoperabilidad** ***(interoperability testing)***
- Buscan problemas de comunicación entre nuestro producto y otros con los que debe trabajar